

honeyOS

User Manual

By Whitespace

Members:

Guarin, Nicolete Rein

Legaspo, Jed Lordy

Pagaran, Niño Christian

Solis, Glen

Table of Contents

I. Introduction and Scope

II. Requirements and Project Setup

III. Build and Boot Procedures

IV. Desktop and Application Navigation

V. File Editing Workflow

VI. Allocation Table and File System Model

VII. Keyboard Reference

VIII. Troubleshooting

IX. Demonstration Flow

I. Introduction and Scope

honeyOS is a small Rust operating system kernel that boots directly in a virtual machine. It uses VGA text mode for screen output and a polled PS/2 keyboard driver for input.

This manual describes the features implemented in the current repository. The details were checked against `src/main.rs`, `src/shell.rs`, `src/keyboard.rs`, `src/fs.rs`, `.cargo/config.toml`, and `scripts/build-virtualbox-disk.sh`.

Important: the current file system is stored in RAM only. Files are lost when the virtual machine is restarted, reset, or powered off.

Two Ways to Run honeyOS

honeyOS can be run in two supported ways:

Method	Use this when	Main requirement
Source code with QEMU	You want to build and boot the OS from the repository source.	Rust nightly setup, bootimage, and QEMU.
Ready-made VDI with VirtualBox	You already have honeyos.vdi and only want to boot the OS.	VirtualBox and the honeyos.vdi disk file.

The source-code path uses `cargo run`, which builds the kernel and starts QEMU through the configured `bootimage runner`. The VDI path does not rebuild the source code; it boots the existing `honeyos.vdi` file as a VirtualBox hard disk.

Current Capabilities

The current build includes:

- a keyboard-driven desktop
- a `File Manager` app
- an `Allocation Table` app
- text file creation
- text file opening and editing
- file saving through `Ctrl+S`
- file renaming through `Ctrl+R`
- file deletion from the file manager
- cursor movement inside the editor
- Caps Lock and Shift support for text entry
- a simulated indexed-allocation file system

Current Boundaries

The current build does not include:

- persistent disk storage for user-created files
- mouse support
- multitasking
- a graphical window system
- an installer ISO

II. Requirements and Project Setup

Host Tools

The required host tools depend on the run method.

For the source-code QEMU path, install:

- Rust and Cargo through `rustup`
- a nightly Rust toolchain
- the `rust-src` component
- the `llvm-tools-preview` component
- `bootimage`
- QEMU

For the ready-made VDI path, install:

- VirtualBox

If the `honeyos.vdi` file must be rebuilt from source, the source-code tools are also required, plus `qemu-img` from QEMU tools.

Check the source-code tools with:

```
rustup --version
cargo --version
qemu-system-x86_64 --version
```

Check the optional VDI tools with:

```
qemu-img --version
VBoxManage --version
```

Rust Toolchain Setup

Run this setup once from the project root:

```
rustup toolchain install nightly
rustup override set nightly
rustup component add rust-src llvm-tools-preview
cargo install bootimage
```

This project requires nightly Rust because `.cargo/config.toml` uses a custom bare-metal target and `build-std`.

Project Root

Use the project root folder, meaning the folder that contains `Cargo.toml`, `Cargo.lock`, `src/`, `docs/`, and `scripts/`.

Move into that folder before running build commands:

```
cd path/to/project-root
```

Replace `path/to/project-root` with the actual project location on the computer being used.

III. Build and Boot Procedures

Choose a Run Path

honeyOS does not use an ISO installer. Use one of these two run paths:

- run from source code with QEMU
- run the existing `honeyos.vdi` file with VirtualBox

Path A: Run from Source Code with QEMU

Use this path when working from the repository source. This path requires the Rust setup from [II. Requirements and Project Setup](#), `bootimage`, and QEMU.

From the project root, run:

```
cargo run
```

Cargo uses `.cargo/config.toml`, which points to the custom `x86_64-honeyos.json` target and the configured `bootimage runner`. The runner starts QEMU with the generated boot image.

QEMU is the reference environment for this project.

To compile without booting:

```
cargo build
```

To create only the bootable raw disk image:

```
cargo bootimage
```

The raw image is created at:

```
target/x86_64-honeyos/debug/bootimage-honeyos.bin
```

Manual QEMU command:

```
qemu-system-x86_64 -drive format=raw,file=target/x86_64-honeyos/debug/bootimage-honeyos.bin
```

Optional: Build or Rebuild the VirtualBox Disk

This step is needed only when `honeyos.vdi` is missing or must be regenerated from the current source code. It is not needed when a ready-made `honeyos.vdi` file is already available.

To create the VirtualBox disk image from source, use the helper script:

```
./scripts/build-virtualbox-disk.sh
```

The script runs `cargo bootimage`, reads:

```
target/x86_64-honeyos/debug/bootimage-honeyos.bin
```

and converts it into:

```
honeyos.vdi
```

The script requires `qemu-img`. It also requires the same Rust and `bootimage` setup used by the source-code QEMU path.

Path B: Run the Ready-Made VDI with VirtualBox

Use this path when the `honeyos.vdi` file already exists and the OS does not need to be rebuilt from source. This path requires VirtualBox and the `honeyos.vdi` disk file. It does not require Rust, Cargo, `bootimage`, QEMU, or `qemu-img` unless the VDI needs to be rebuilt.

Create the VirtualBox Machine

In VirtualBox, boot from `honeyos.vdi` as a hard disk. Do not select an ISO image.

1. Open VirtualBox.
2. Click `New`.
3. Use a VM name such as `honeyOS`.
4. Leave the ISO image field empty.
5. Set `OS` to `Other`.

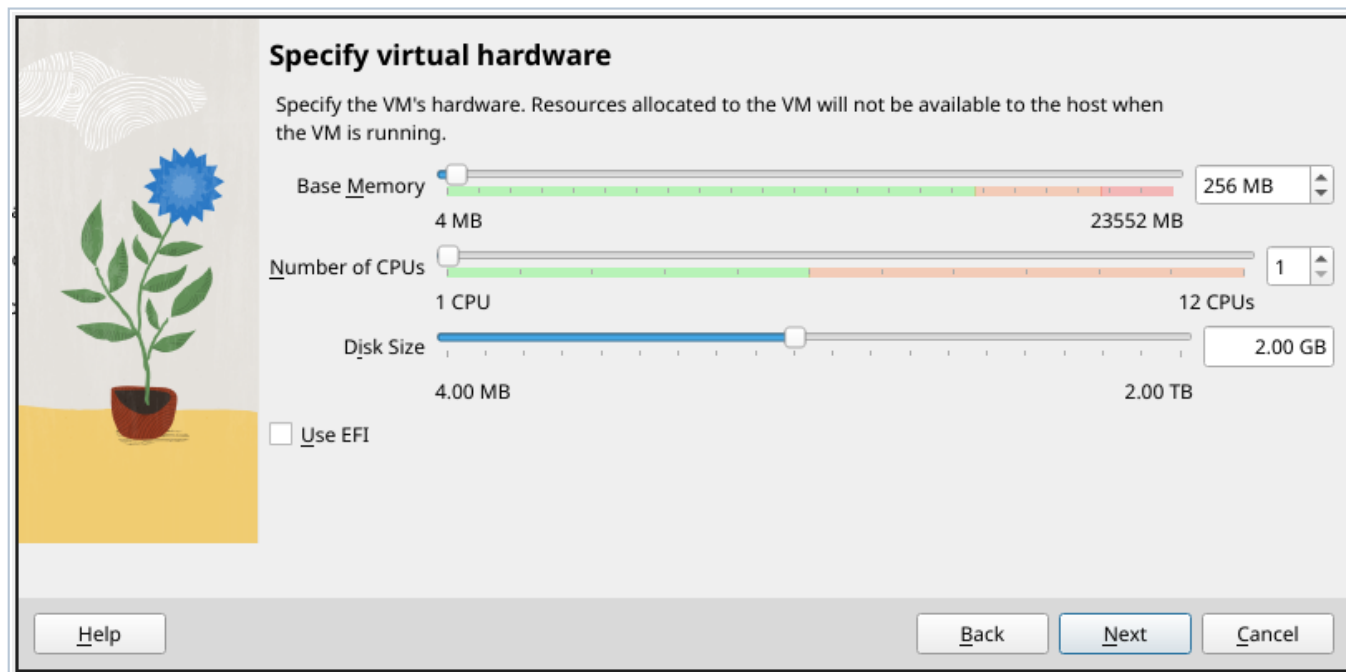
6. Set **OS Version** to **Other/Unknown (64-bit)**.
7. Continue to the hardware screen.

Create the VM without an ISO image and use an **Other/Unknown (64-bit)** guest type.

Configure Virtual Hardware

Recommended VM settings:

- Base Memory: **128 MB** minimum; **256 MB** is fine
- CPUs: **1**
- EFI: disabled
- Boot mode: legacy BIOS



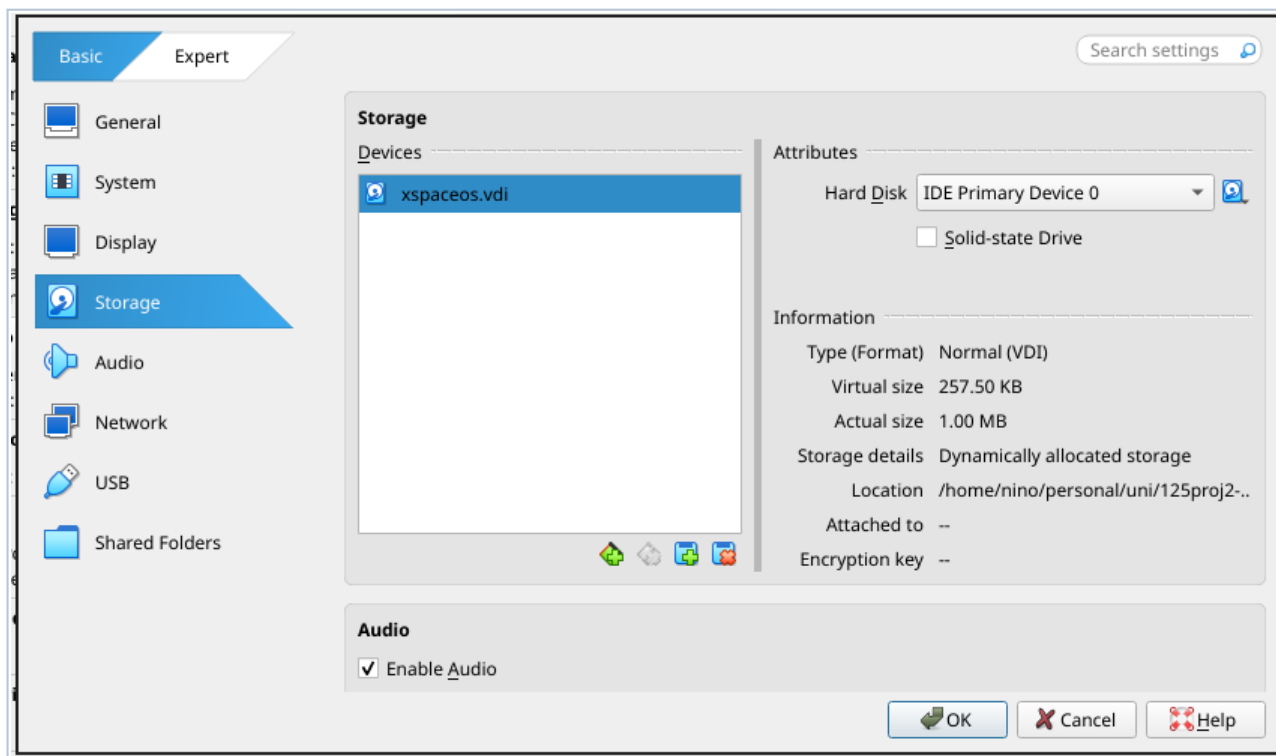
Caption: Keep **Use EFI** unchecked. The current boot image expects legacy BIOS boot.

VirtualBox may ask for a virtual hard disk size during VM creation. It is fine to finish the wizard, because **honeynos.vdi** will be attached in the next step.

Attach `honeynos.vdi`

After obtaining **honeynos.vdi** :

1. Select the VM in VirtualBox.
2. Open **Settings**.
3. Open **Storage**.
4. Remove any temporary blank disk if VirtualBox created one.
5. Attach **honeynos.vdi** as the VM hard disk.
6. Save the settings.



Caption: Attach the `honeynos.vdi` file as the VM's hard disk.

Boot the OS in VirtualBox

Start the VM. A successful boot shows the honeyOS desktop.



Caption: The desktop shows the `File Manager` and `Allocation Table` apps.

IV. Desktop and Application Navigation

Desktop Screen

The desktop is the first screen after boot.

Desktop controls:

Key	Action
Up / Down	Move the highlighted selection.
Enter	Open the selected app.

Available apps:

App	Purpose
File Manager	Create, open, edit, rename, and delete files.
Allocation Table	Inspect simulated file allocation blocks.

File Manager Screen

Open **File Manager** from the desktop by leaving it highlighted and pressing **Enter**.

The file manager shows:

- **[+ New File]** at the top
- existing files below it
- each file's current size in bytes



Caption: The file manager lists files and shows each file's byte count.

File manager controls:

Key	Action
Up / Down	Move the highlighted selection.
Enter on [+ New File]	Start creating a new file.
Enter on a file	Open that file in the editor.
Delete on a file	Delete the selected file immediately.
Esc	Return to the desktop.

There is no confirmation prompt before deleting a file. Because the file system is currently in memory only, deleted files cannot be recovered.

V. File Editing Workflow

Create a File

1. Open `File Manager`.
2. Highlight `[ + New File ]`.
3. Press `Enter`.
4. Type the file name at the `File name:` prompt.
5. Press `Enter`.

If the name is accepted, the editor opens automatically for the new file.

File name rules:

- the name cannot be empty
- the name cannot be longer than `32` bytes
- the name cannot match an existing file

The OS reports an error if the file table is full or the name already exists. If an empty name is entered, the OS returns to the file manager without creating a file.

Open a File

1. Open `File Manager`.
2. Highlight the file name.
3. Press `Enter`.

There is no separate read-only viewer. Files are opened in the editor.

Edit Text

The editor is a full-screen text editor. It shows:

- the current file name in the top bar
- editor shortcuts in the top bar
- file contents in the main area
- saved/unsaved status in the bottom bar
- current size out of the `512` byte limit
- `CAPS` in the bottom bar when Caps Lock is enabled



Caption: The editor supports typing, cursor movement, deletion, saving, closing, and renaming.

Editor controls:

Key	Action
Printable keys	Insert text at the cursor.
Enter	Insert a newline.
Backspace	Delete the character before the cursor.
Delete	Delete the character at the cursor.
Left / Right	Move the cursor one character left or right.
Up / Down	Move through visual editor rows.
Home	Move to the start of the current visual row.
End	Move to the end of the current visual row.
PageUp / PageDown	Move up or down by one visible editor page.
Ctrl+S	Save the file.
Ctrl+R	Rename the file.
Ctrl+X	Close the editor and return to the file manager.

Save before closing. `Ctrl+X` does not ask whether unsaved changes should be kept.

Save a File

Press `Ctrl+S` inside the editor.

When saving succeeds, the bottom status bar changes to `Saved`. Editing after that changes the status back to `Unsaved`.

The editor stops accepting more text after the file reaches `512` bytes.

Rename a File

1. Open the file in the editor.
2. Press **Ctrl+R**.
3. Type the new name at the **New name:** prompt.
4. Press **Enter**.



```
=== Rename File ===  
  
Current name: hello  
New name: namename
```

Caption: Renaming is done from inside the editor.

Rename rules:

- the new name cannot be empty
- the new name cannot be longer than **32** bytes
- the new name cannot match an existing file

Renaming changes only the file name. It does not change the file contents or the file's allocated data blocks.

Delete a File

1. Open **File Manager**.
2. Highlight the file.
3. Press **Delete**.

The selected file is removed immediately. Its index block and data blocks are returned to the free block pool.

VI. Allocation Table and File System Model

Allocation Table Screen

Open **Allocation Table** from the desktop to inspect the simulated file allocation state.

The table displays block entries in two columns. Each row contains:

- block number, such as **B00**
- state: **USED** or **FREE**
- block type: **IDX**, **DAT**, or **---**
- owner file name, if the block is used

```

B00 USED  IDX namename          B10 FREE  --- -
B01 USED  DAT namename          B11 FREE  --- -
B02 USED  IDX testfile          B12 FREE  --- -
B03 USED  DAT testfile          B13 FREE  --- -
B04 FREE  --- -                 B14 FREE  --- -
B05 FREE  --- -                 B15 FREE  --- -
B06 FREE  --- -                 B16 FREE  --- -
B07 FREE  --- -                 B17 FREE  --- -
B08 FREE  --- -                 B18 FREE  --- -
B09 FREE  --- -                 B19 FREE  --- -

```

Left/Right = page Esc = desktop Page 1/2

Caption: A file uses one `IDX` block and one or more `DAT` blocks after content is saved.

Allocation table controls:

Key	Action
Left / Right	Move between allocation-table pages.
Esc	Return to the desktop.

The current file system has `40` simulated blocks, so the allocation table has two pages of `20` blocks each.

Indexed Allocation Model

honeyOS currently uses a fixed-size, in-memory file system. It is designed for a small `no_std` kernel, so it does not require heap allocation.

Storage model:

- files are kept in kernel RAM
- each file reserves one index block when created
- file contents are split into `128` byte data blocks when saved
- each file can own up to four data blocks
- deleting a file frees its index block and data blocks
- saving a file replaces its previous data-block allocation

After deleting a file, its blocks become `FREE` again:

```
B00 USED  IDX  namename          B10 FREE  ---  -
B01 USED  DAT  namename          B11 FREE  ---  -
B02 FREE  ---  -                  B12 FREE  ---  -
B03 FREE  ---  -                  B13 FREE  ---  -
B04 USED  IDX  colete            B14 FREE  ---  -
B05 USED  DAT  colete            B15 FREE  ---  -
B06 FREE  ---  -                  B16 FREE  ---  -
B07 FREE  ---  -                  B17 FREE  ---  -
B08 FREE  ---  -                  B18 FREE  ---  -
B09 FREE  ---  -                  B19 FREE  ---  -

eft/Right = page  Esc = desktop  Page 1/2
```

File System Limits

Current limits:

Limit	Value
Maximum files	8
Maximum file name length	32 bytes
Maximum file content length	512 bytes
Block size	128 bytes
Maximum data blocks per file	4
Index blocks per file	1
Total simulated blocks	40
Persistence	None; RAM only

What this means for users:

- Empty files still use one index block.
- A saved file with 1 to 128 bytes uses one data block.
- A saved file with 129 to 256 bytes uses two data blocks.
- A saved file with 257 to 384 bytes uses three data blocks.
- A saved file with 385 to 512 bytes uses four data blocks.
- Files disappear after reboot or power off.

VII. Keyboard Reference

Text input supports standard US keyboard scancode set 1 behavior for the keys implemented by the current driver.

Supported behavior:

- letters, numbers, spaces, and common punctuation

- **Shift** for uppercase letters
- **Caps Lock** for alphabetic capitalization
- **Shift** with the number row, such as **!**, **@**, **#**, **\$**, **%**, **^**, **&**, *****, **(**, and **)**
- **Shift** with punctuation, such as **_**, **+**, **{**, **}**, **|**, **:**, **"**, **~**, **<**, **>**, and **?**
- left and right arrow keys
- **Home**, **End**, **PageUp**, **PageDown**
- **Backspace**, **Delete**, **Enter**, and **Esc**
- **Ctrl+S**, **Ctrl+R**, and **Ctrl+X** in the editor

Notes:

- Caps Lock affects letters only.
- Holding Shift while Caps Lock is active types lowercase letters.
- Unsupported key combinations are ignored.
- The OS does not use mouse input.

VIII. Troubleshooting

`cargo build` reports `build-std` or target errors

Make sure nightly Rust is active and the required components are installed:

```
rustup override set nightly
rustup component add rust-src llvm-tools-preview
```

`cargo bootimage` is not found

Install **bootimage**:

```
cargo install bootimage
```

`qemu-img` is not found

Install QEMU tools, then verify:

```
qemu-img --version
```

VirtualBox asks for an ISO

Do not select an ISO. honeyOS boots from **honeynos.vdi**.

Use this process:

1. Create the VM without installation media.
2. Finish the VM wizard.
3. Open **Settings > Storage**.
4. Attach **honeynos.vdi** as the hard disk.

VirtualBox does not boot the OS

Check the following:

- **honeynos.vdi** exists in the project root
- the VDI was rebuilt after the latest code changes
- the VDI is attached as the VM hard disk
- EFI is disabled
- the VM type is a 64-bit **Other/Unknown** style guest

- the raw image boots with QEMU

If QEMU boots but VirtualBox does not, use QEMU for the demo. QEMU is the reference platform for this project.

Files disappear after restarting

This is expected in the current implementation. The file system is in memory only and does not write user files to the VDI.

IX. Demonstration Flow

Use this short flow for a clean demonstration. First boot honeyOS by using one of the two run paths: run `cargo run` for the source-code QEMU path, or start the VirtualBox VM that has `honeyos.vdi` attached.

1. Open `File Manager`.
2. Create a file named `notes.txt`.
3. Type:

```
honeyOS demo file
Created inside the OS editor
Files are currently RAM-only
```

1. Press `Ctrl+S` to save.
2. Press `Ctrl+R` and rename the file to `manual.txt`.
3. Press `Ctrl+X` to return to the file manager.
4. Confirm the renamed file appears with its byte count.
5. Press `Esc` to return to the desktop.
6. Open `Allocation Table`.
7. Show the `IDX` block and `DAT` block entries for the file.

This demonstrates booting, file management, editing, saving, renaming, and indexed allocation inspection.